

Subprograms: Variable Arguments

Programming Languages

Millersville University

Outline

- Definition
- Case Studies
 - C
 - C++
 - Java
 - JavaScript
 - Python/Ruby

Variable Number of Arguments

- Some functions should be able to accept *any* number of arguments, not just a limited number

What classes of functions should support this?

Variable Arguments: C

- Oldest language that supports variable arguments
- An afterthought – an extension to the language (`stdarg.h`); so the mechanism is low-level and manual
- All variable arguments must come at the end; n indicates # of arguments

```
int sum (int n, ...) {  
    va_list args;  
    va_start (args, n);  
    int s = 0;  
    for (int i = 0; i < n; ++i) {  
        s += va_arg (args, int);  
    }  
    va_end (args);  
    return s;  
}
```

Implemented via
assistive macros:

- `va_list`
- `va_start`
- `va_arg`
- `va_end`

Variable Arguments: C++

- Template argument `typename... Ts` is a template parameter pack
- Argument `Ts... ts` is a function parameter pack
- Expression `(0 + ... + ts)` is a fold expression (left)
- `sum(1, 2.5, 3)` $\rightarrow$ `Ts = {int, double, int}`; `ts...` holds all args

```
template<typename... Ts>
auto sum (Ts... ts) {
    return (0 + ... + ts);
}
```

Variable Arguments: Java

- Java allows user to specify any number of args with a triple-dot syntax
- Named formal parameter acts like an **array**
- Variable argument must come at the end

```
int sum (int... values) {  
    int s = 0;  
    for (int v : values) {  
        s += v;  
    }  
    return s;  
}
```

Variable Arguments: JavaScript

- JavaScript allows user to specify any number of args with a triple-dot syntax
- Named formal parameter acts like an **array**
- No parameters may come after variable argument

```
function sum (... values) {  
    return values.reduce ((s, v) => s + v, 0);  
}
```

Variable Arguments: Python/Ruby

- Both Python and Ruby offer a special syntax for a parameter that allows for any number of args
- An **asterisk** denotes that a formal parameter corresponds to zero or more actual parameters (as an iterable)
- A **double asterisk** denotes that a formal parameter corresponds to zero or more named actual parameters (as a dictionary)
- Strict Ordering Requirement
 - positional parameters
 - *args (variable positional params)
 - **kwargs (keyword params)

Variable Arguments: Python/Ruby

- An **asterisk** denotes that a formal parameter corresponds to zero or more actual parameters

```
def mysum(*nums):  
    s = 0  
    for v in nums:  
        s += v  
    return s
```

```
def mysum(*nums)  
    s = 0  
    nums.each do |v|  
        s += v  
    end  
    return s  
end
```

Variable Arguments: Python

- A **double asterisk** denotes that a formal parameter corresponds to zero or more named actual parameters (as a dictionary/map)

```
def my_arg_checker(**args):  
    if "magic" in args:  
        print (f'magic={args["magic"]}')  
        return True  
    return False
```

```
my_arg_checker (a=4)    // args = {"a": 4}  
my_arg_checker (b=3, magic=42) // args = {"b":3,  
"magic": 42}
```

Attribution

Slides originally developed by William Killian,
modified by Stephanie Schwartz

modified by Gary Zoppetti

modified by Jingnan Xie

Sources include Robert Sebesta's [Concepts of Programming Languages](#)